Université de Rouen Normandie

Master 1 – Génie Informatique et Logiciel

Projet Gestion de Flotte

Rapport de Semaine 6

Architecture Frontend & Sécurité (SSO)

Étudiants :
Ahcène Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Année universitaire :
2025 – 2026

Semaine :
6 / 7

16 avril 2026

Table des matières

1 Introduction **2**

2 Semaine 6 : Architecture Frontend et Sécurité (SSO) **2**

2.1 Objectifs et Philosophie d'Architecture 2
2.2 Structure en Monorepo (NPM Workspaces) 2
2.3 Approche Micro-Frontend via Module Federation 2
2.4 Sécurité Centralisée et Single Sign-On (SSO) avec Keycloak 3
2.5 Conception du Package Transversal : AuthProvider 3
2.6 Routage Global et Chargement Asynchrone (Shell) 3
2.7 Communication Inter-Services : Client GraphQL Partagé 3
2.8 Déploiement et Intégration du Micro-Frontend "Véhicules" 3
2.9 Commandes Fondamentales de l'Infrastructure 4
2.10 Intégration et Consommation du Contrat GraphQL 4
2.11 Méthodologie d'implémentation : Cas du module "Conducteurs" 4
2.11.1 Initialisation et dépendances 5
2.11.2 Configuration de la Fédération 5
2.11.3 Définition du Contrat GraphQL 5
2.11.4 Intégration dans le Shell 5

3 Conclusion et Perspectives de Finalisation **6**

1. Introduction

La semaine 6 marque une étape décisive dans le développement de la plateforme de gestion de flotte avec la mise en place de l'infrastructure frontend. L'objectif principal a été de concevoir une architecture capable de supporter la croissance du projet tout en garantissant une sécurité robuste et centralisée.

L'approche retenue s'appuie sur le concept de **Micro-Frontends** et l'utilisation d'un **Monorepo**, permettant une séparation stricte des responsabilités et une scalabilité organisationnelle optimale.

2. Semaine 6 : Architecture Frontend et Sécurité (SSO)

2.1. Objectifs et Philosophie d'Architecture

Pour répondre aux exigences de scalabilité et de maintenabilité, l'interface utilisateur n'a pas été conçue comme une application monolithique, mais selon une architecture orientée composants isolés. Cette approche s'articule autour de deux concepts majeurs : le **Monorepo** et les **Micro-Frontends (MFE)**.

L'objectif est de permettre à plusieurs développeurs d'opérer en parallèle sur des modules distincts (ex : cartographie, gestion des véhicules) sans générer de conflits, tout en garantissant une cohérence visuelle et sécuritaire centralisée.

2.2. Structure en Monorepo (NPM Workspaces)

L'écosystème frontend est regroupé au sein d'un dépôt unique organisé via les **NPM Workspaces**. Cette structure permet de mutualiser les dépendances globales tout en séparant strictement le code source. L'arborescence se divise en deux catégories principales :

— **Le répertoire `apps/` :** Contient les applications exécutables de manière autonome (ex : `shell`, `location`, `vehicles`).

— **Le répertoire `packages/` :** Regroupe le code transversal partagé entre les applications (ex : `shared-ui` pour les composants graphiques, `shared-auth` pour la sécurité).

2.3. Approche Micro-Frontend via Module Federation

L'assemblage des différentes interfaces s'effectue dynamiquement côté client grâce au plugin **Module Federation** de l'outil de build Vite.js.

— **L'application Hôte (Shell) :** Elle agit comme une coquille vide de haut niveau. Son rôle se limite à gérer le routage global, l'interface racine (barre de navigation) et l'application stricte de la politique de sécurité.

— **Les applications Distantes (Remotes) :** Ce sont des micro-applications indépendantes. Au moment de la navigation, le Shell télécharge et injecte dynamiquement (via `React.lazy` et `Suspense`) le composant exposé par le Remote.

2.4. Sécurité Centralisée et Single Sign-On (SSO) avec Keycloak

Afin de garantir un niveau de sécurité optimal, le processus d'authentification a été délégué au standard **OAuth2 / OpenID Connect (OIDC)** via un serveur **Keycloak** conteneurisé.

> **Flux d'authentification :** Aucun formulaire de connexion n'est implémenté dans l'application React. Les requêtes non authentifiées sont redirigées vers Keycloak (Standard Flow avec PKCE). Après validation, Keycloak restitue un jeton JWT permettant aux applications frontend d'habiliter l'accès en fonction des rôles (RBAC).

2.5. Conception du Package Transversal : AuthProvider

La mécanique d'interception et de vérification du jeton OIDC a été encapsulée dans le paquetage `@flotte/shared-auth`. Ce module expose un Provider de contexte React injecté à la racine de l'application Shell.

1. **Verrou d'initialisation (Anti-Race Condition) :** Un verrou mémorisé hors de la portée du composant garantit une unique instanciation du client Keycloak, même en environnement de développement (React Strict Mode).
2. **Blocage du rendu :** Le Provider bloque le montage de l'arbre DOM tant que l'initialisation de Keycloak n'est pas résolue, empêchant l'affichage momentané de routes protégées (*Flickering*).
3. **Hook personnalisé (`useAuth`) :** Expose une API de faąde permettant à n'importe quel micro-frontend de récupérer l'identité de l'utilisateur ou de déclencher une déconnexion.

2.6. Routage Global et Chargement Asynchrone (Shell)

L'application hôte (`Shell`) orchestre la navigation grâce à `react-router-dom`. Afin d'optimiser les performances, les modules distants ne sont pas chargés au démarrage mais importés dynamiquement via `React.lazy()`. Ce mécanisme de *Lazy Loading* diffère le téléchargement des bundles JavaScript jusqu'au moment précis où l'utilisateur accède à la route correspondante.

2.7. Communication Inter-Services : Client GraphQL Partagé

Pour connecter l'interface à la Gateway GraphQL, un second paquetage partagé a été développé : `@flotte/shared-client`, basé sur `@apollo/client`.

Grâce à un intercepteur réseau (*Context Link Apollo*), le client récupère dynamiquement le jeton JWT fourni par l'`AuthProvider` et l'injecte dans l'en-tête HTTP `Authorization: Bearer <token>` de chaque requête sortante.

2.8. Déploiement et Intégration du Micro-Frontend "Véhicules"

Le développement du module de gestion de flotte (CRUD Véhicules) valide l'approche fédérée. C'est un projet Vite.js autonome qui consomme l'API GraphQL pour

afficher les données métiers.

Configuration technique :

- **Allocation de ports :** Utilisation de ports TCP statiques (ex : 5002) pour éviter les conflits CORS.
- **Dependency Sharing :** Partage des bibliothèques critiques (`react`, `apollo`) comme singletons pour éviter le double chargement et les conflits de Hooks.

2.9. Commandes Fondamentales de l'Infrastructure

La gestion du Monorepo NPM et le cycle de construction des Micro-Frontends s'articulent autour des commandes suivantes :

```
# Creation d'une nouvelle sous-application
npm create vite@latest nom_app -- --template react-ts

# Installation du plugin de federation
npm install @originjs/vite-plugin-federation --save-dev

# Compilation isolee du module distant
npm run build --workspace=nom_app

# Exposition locale du module compile (preview)
npm run preview --workspace=nom_app

# Demarrage du Shell principal
npm run dev --workspace=shell
```

Listing 1 – Gestion du Monorepo et des MFEs

2.10. Intégration et Consommation du Contrat GraphQL

La traduction du schéma GraphQL en composants React s'articule autour de quatre axes :

- **Typage statique :** Les types GraphQL (ex : `Vehicle`) sont transposés en interfaces TypeScript.
- **Optimisation réseau :** Utilisation de la syntaxe `gql` pour ne réclamer que les champs nécessaires (*No Over-fetching*).
- **Rendu piloté par les Enums :** Utilisation des `VehicleStatus` pour conditionner dynamiquement le rendu visuel.
- **Abstraction du cycle de vie (`useQuery`) :** Gestion native des états de chargement et de mise en cache.

2.11. Méthodologie d'implémentation : Cas du module "Conducteurs"

L'ajout d'un nouveau domaine métier suit un processus standardisé :

2.11.1. Initialisation et dépendances

```
npm create vite@latest drivers -- --template react-ts
cd drivers
npm install @apollo/client graphql
npm install @originjs/vite-plugin-federation --save-dev
```

2.11.2. Configuration de la Fédération

```
export default defineConfig({
  plugins: [
    react(),
    federation({
      name: 'driversapp',
      filename: 'remoteEntry.js',
      exposes: {
        './DriverList': './src/App.tsx',
      },
      shared: ['react', 'react-dom', '@apollo/client', 'graphql'],
    }),
  ],
  server: { port: 5003, strictPort: true }
});
```

Listing 2 – drivers/vite.config.ts (extraits)

2.11.3. Définition du Contrat GraphQL

```
export const GET_DRIVERS = gql`
  query GetDrivers {
    drivers {
      items {
        id
        firstname
        lastname
        status
      }
    }
  }
`;
```

Listing 3 – drivers/src/queries.ts

2.11.4. Intégration dans le Shell

```
remotes: {
  driversapp: 'http://localhost:5003/assets/remoteEntry.js',
}
```

Listing 4 – shell/vite.config.ts

```
const DriverList = React.lazy(() => import('driversapp/DriverList'));

<Route path="drivers" element={
  <Suspense fallback={<div>Chargement du module...</div>}>
    <DriverList />
  </Suspense>
} />
```

Listing 5 – shell/App.tsx

3. Conclusion et Perspectives de Finalisation

Bien que l'ossature technique des Micro-Frontends et la sécurité centralisée soient désormais opérationnelles, certains modules nécessitent encore des ajustements finaux.

> **Note sur le rendu :** La partie relative à la **cartographie interactive** ainsi que les dernières optimisations de l'interface sont en cours de finalisation. Afin de fournir un livrable complet et un rapport de qualité supérieure, l'ensemble du projet (code source et documentation finale) sera officiellement soumis ce **samedi**.

Ce délai supplémentaire nous permet d'assurer une intégration parfaite entre le service de localisation (backend) et son rendu visuel (frontend), tout en affinant la cohérence globale de la plateforme.